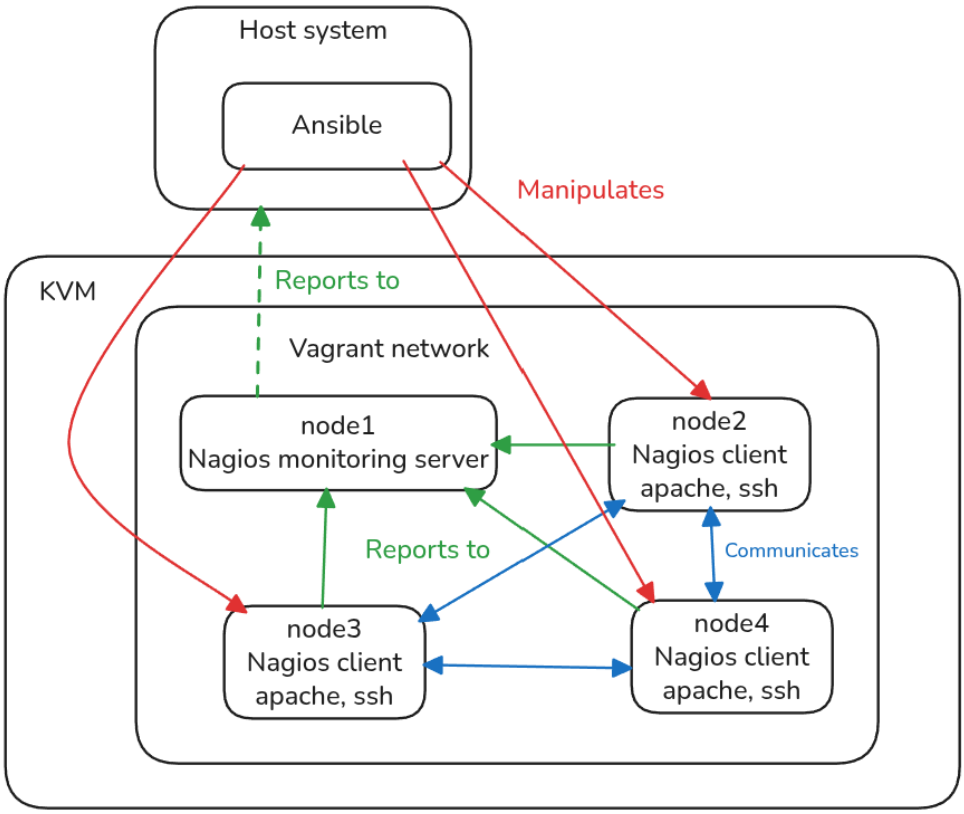
CPT_S 427 Project – Ansible/Nagios systems management
Dante Meyer

Summary

I assembled a network of Ubuntu virtual machines using the *Vagrant* command line utility on a Fedora Linux host machine, and orchestrated the systems with Ansible. Ansible is a utility that allowed me to install packages, run commands, and alter files through easily replicable "playbooks", that apply idempotent changes to target systems. Vagrant has built-in integration with Ansible, making this my platform of choice.

To monitor the systems deployed with Vagrant/Ansible, I used Nagios 4 Core, an open-source network monitoring system. Three nodes (named *node2*, *node3*, and *node4*) have a Nagios client program, which reports data at request to the first node (named *node1*), which has the Nagios server program. The Nagios server requests data from the clients at regular intervals, to monitor network and system health. The server is accessible through a web interface, accessible at *http://[node1_ip]/nagios4/*. I configured the interface to require authentication, which is necessary to enable certain features in Nagios. A system diagram is included below:



Using Ansible and several Bash scripts that call it, I performed the following tasks:

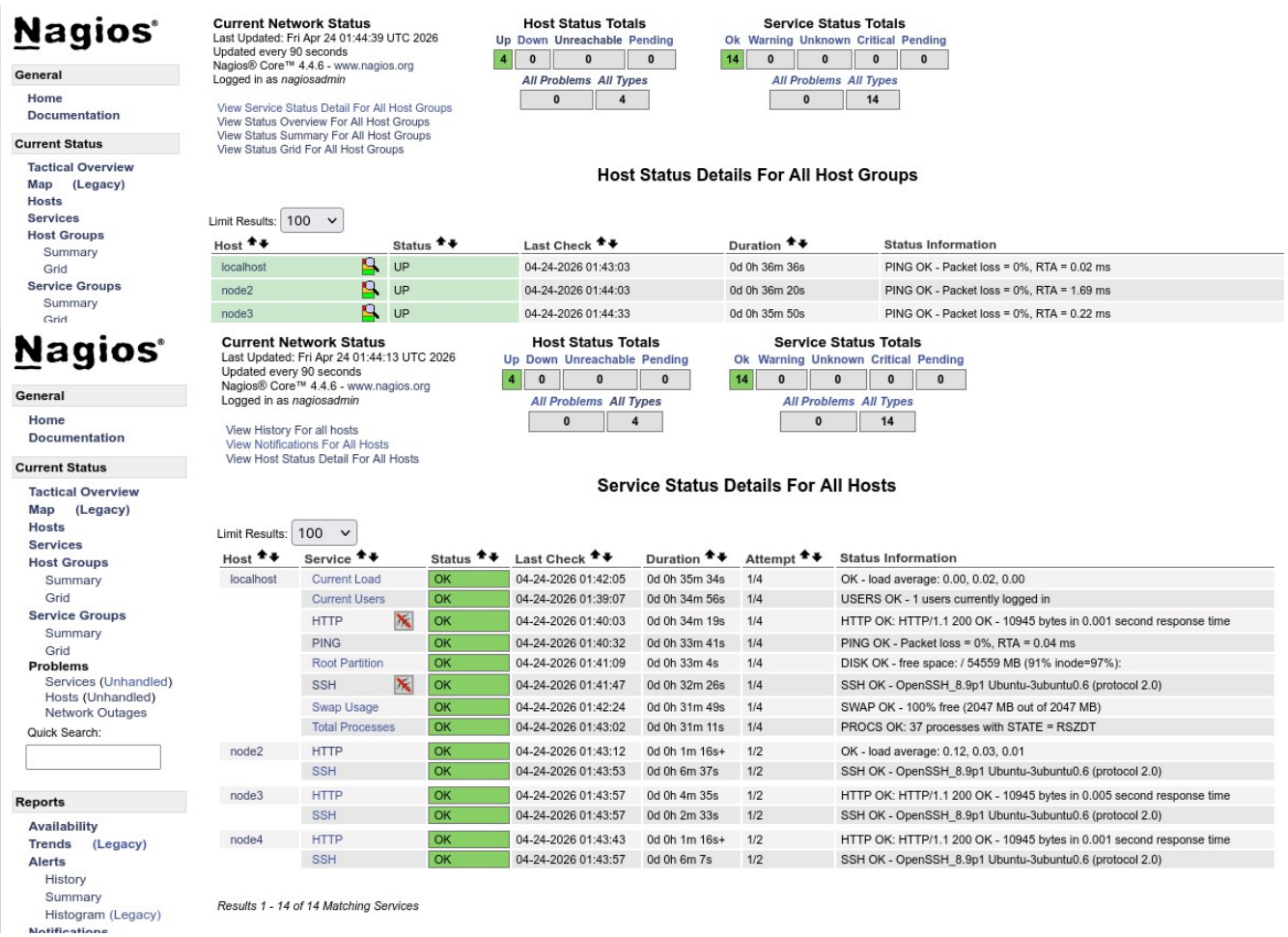
1. Set up Nagios, monitor the initial (healthy) state of the network.
2. Inject faults with the script *inject_faults.sh*, which calls an Ansible playbook that manipulates the Vagrant client machines.
3. Monitor the reported activity by Nagios

4. Recover the faults using the script *recover_faults.sh*, which calls the same Ansible playbook, but only tasks that reverse the fault injections.
5. Monitor the resulting state in Nagios.

Results

Initial State

When capturing the system's initial state via Nagios, all the hosts and monitored services (namely, HTTP and SSH) were listed as healthy.



The availability reported by Nagios was 100%, not including periods where Nagios was still being set up. After waiting a few minutes to build initial availability data on the machines, I ran the fault injection script.

Injecting Faults

The fault injection script I ran does the following, targeting different nodes:

- Adds 75% packet loss to node 3, creating an unstable network connection.

CPT_S 427 Project – Ansible/Nagios systems management
Dante Meyer

- Adds 100% packet loss to node 2
- Adds a 5 second delay with 3 seconds of jitter, to all nodes.
- Disables SSH for all nodes, with a recovery scheduled after 2 minutes.
- Disables Apache2 for all nodes.

I chose these faults because they could easily arise from hardware or software errors, affect the network's health as a whole, and have support for monitoring by Nagios. All the faults are injected at the same time. Since I configured Nagios to poll for faults relatively quickly (once per minute, with 2 retries), it picked up on the faults within a minute. It was *node3* that was first marked as "DOWN".

Current Network Status
Last Updated: Fri Apr 24 02:02:05 UTC 2026
Updated every 90 seconds
Nagios® Core™ 4.4.6 - www.nagios.org
Logged in as *nagiosadmin*

View Service Status Detail For All Host Groups
View Status Overview For All Host Groups
View Status Summary For All Host Groups
View Status Grid For All Host Groups

Host Status Totals

Up	Down	Unreachable	Pending
3	1	0	0

All Problems	All Types
1	4

Service Status Totals

Ok	Warning	Unknown	Critical	Pending
8	0	0	6	0

All Problems	All Types
6	14

Host Status Details For All Host Groups

Limit Results: 100

Host	Status	Last Check	Duration	Status Information
localhost	UP	04-24-2026 01:58:03	0d 0h 54m 2s	PING OK - Packet loss = 0%, RTA = 0.03 ms
node2	UP	04-24-2026 02:01:42	0d 0h 53m 46s	PING OK - Packet loss = 80%, RTA = 4114.23 ms
node3	DOWN	04-24-2026 02:01:03	0d 0h 1m 2s	PING CRITICAL - Packet loss = 83%, RTA = 5989.75 ms
node4	UP	04-24-2026 02:01:21	0d 0h 52m 46s	PING OK - Packet loss = 75%, RTA = 3287.31 ms

Results 1 - 4 of 4 Matching Hosts

After a short period of time, more hosts began to fail:

Current Network Status
Last Updated: Fri Apr 24 02:03:10 UTC 2026
Updated every 90 seconds
Nagios® Core™ 4.4.6 - www.nagios.org
Logged in as *nagiosadmin*

View Service Status Detail For All Host Groups
View Status Overview For All Host Groups
View Status Summary For All Host Groups
View Status Grid For All Host Groups

Host Status Totals

Up	Down	Unreachable	Pending
2	2	0	0

All Problems	All Types
2	4

Service Status Totals

Ok	Warning	Unknown	Critical	Pending
9	0	0	5	0

All Problems	All Types
5	14

Host Status Details For All Host Groups

Limit Results: 100

Host	Status	Last Check	Duration	Status Information
localhost	UP	04-24-2026 02:03:03	0d 0h 55m 7s	PING OK - Packet loss = 0%, RTA = 0.07 ms
node2	DOWN	04-24-2026 02:02:42	0d 0h 0m 28s	PING CRITICAL - Packet loss = 83%, RTA = 5230.62 ms
node3	UP	04-24-2026 02:02:09	0d 0h 1m 1s	PING OK - Packet loss = 75%, RTA = 2978.53 ms
node4	DOWN	04-24-2026 02:02:21	0d 0h 0m 49s	PING CRITICAL - Packet loss = 83%, RTA = 5849.64 ms

Results 1 - 4 of 4 Matching Hosts

CPT_S 427 Project – Ansible/Nagios systems management
Dante Meyer

Nagios also reported critical errors for multiple targeted services.

Current Network Status
Last Updated: Fri Apr 24 02:02:51 UTC 2026
Updated every 90 seconds
Nagios® Core™ 4.4.6 - www.nagios.org
Logged in as *nagiosadmin*

View History For all hosts
View Notifications For All Hosts
View Host Status Detail For All Hosts

Host Status Totals

Up	Down	Unreachable	Pending
3	1	0	0

All Problems	All Types
1	4

Service Status Totals

Ok	Warning	Unknown	Critical	Pending
9	0	0	5	0

All Problems	All Types
5	14

Service Status Details For All Hosts

Limit Results: 100

Host	Service	Status	Last Check	Duration	Attempt	Status Information
localhost	Current Load	OK	04-24-2026 02:02:05	0d 0h 54m 12s	1/4	OK - load average: 0.05, 0.05, 0.01
	Current Users	OK	04-24-2026 01:59:07	0d 0h 53m 34s	1/4	USERS OK - 0 users currently logged in
	HTTP	OK	04-24-2026 02:00:03	0d 0h 52m 57s	1/4	HTTP OK: HTTP/1.1 200 OK - 10945 bytes in 0.004 second response time
	PING	OK	04-24-2026 02:00:32	0d 0h 52m 19s	1/4	PING OK - Packet loss = 0%, RTA = 0.05 ms
	Root Partition	OK	04-24-2026 02:01:09	0d 0h 51m 42s	1/4	DISK OK - free space: / 54559 MB (91% inode=97%):
	SSH	OK	04-24-2026 02:01:47	0d 0h 51m 4s	1/4	SSH OK - OpenSSH_8.9p1 Ubuntu-3ubuntu0.6 (protocol 2.0)
	Swap Usage	OK	04-24-2026 02:02:24	0d 0h 50m 27s	1/4	SWAP OK - 100% free (2046 MB out of 2047 MB)
	Total Processes	OK	04-24-2026 01:58:02	0d 0h 49m 49s	1/4	PROCS OK: 35 processes with STATE = RSZDT
node2	HTTP	CRITICAL	04-24-2026 02:01:32	0d 0h 1m 19s	1/2	CHECK_NRPE STATE CRITICAL: Socket timeout after 10 seconds.
	SSH	CRITICAL	04-24-2026 02:01:56	0d 0h 0m 55s	2/2	connect to address 192.168.50.12 and port 22: Connection refused
node3	HTTP	CRITICAL	04-24-2026 02:02:07	0d 0h 1m 44s	1/2	connect to address 192.168.50.13 and port 80: Connection refused
	SSH	OK	04-24-2026 02:02:03	0d 0h 0m 48s	1/2	SSH OK - OpenSSH_8.9p1 Ubuntu-3ubuntu0.6 (protocol 2.0)
node4	HTTP	CRITICAL	04-24-2026 02:02:03	0d 0h 3m 58s	2/2	connect to address 192.168.50.14 and port 80: Connection refused
	SSH	CRITICAL	04-24-2026 02:02:21	0d 0h 0m 30s	2/2	CRITICAL - Socket timeout after 10 seconds

As time went on, several nodes were reached by Nagios, and their state was marked as “UP”, only for the same nodes to disconnect after an arbitrary time span. After multiple reconnects and disconnects, Nagios noticed the rapid state change of some nodes (with a 20% threshold in state changes), and marked them as “flapping”, providing an appropriate warning:

Entry Time	Author	Comment	Comment ID	Persistent	Type	Expires	Action
04-24-2026 02:05:19	(Nagios Process)	Notifications for this host are being suppressed because it was detected as having been flapping between different states (23.0% change > 20.0% threshold). When the host state stabilizes and the flapping stops, notifications will be re-enabled.	1	No	Flap Detection	N/A	

Nagios also provides a metric for overall system health, which was marked at 64% during the fault period:

Network Health

Host Health:

Service Health:

CPT_S 427 Project – Ansible/Nagios systems management
Dante Meyer

Recovery

To recover the systems, I ran *recover_faults.sh*, which ran Ansible tasks that reverse all injected faults. Nagios then reported all hosts as “UP”, and later discarded the state change alerts. The system health was brought back to 91%. A few services, like *node3*’s HTTP service were initially marked as “DOWN”, but were later brought up after Nagios ran a subsequent check. The overall availability report listed the hosts as having an 89% availability rate.

Analysis

While it may often be overlooked, availability is a fundamental aspect of cybersecurity. External adversaries often stage attacks on distributed systems to render them unavailable to their users. With the rise of Software-as-a-Service systems and cloud computing, both of which rely heavily on distributed systems management, this can cause hundreds of derivative applications to go offline or become unusable. When this happens, the parties involved can be hurt through finance or reputation. Pivotal systems can also be affected, like power plants and cloud storage providers that form the backbone of modern online infrastructure, giving attackers more than enough motive to exploit vulnerabilities that affect system availability.

When a vulnerability is found and exploited, steps must be taken to mitigate the cascading effects of attacks. Tools like Nagios help with the first step, identifying gaps in availability, providing reports to system administrators that indicate their time spans and causes. Even without a direct link to a cause, a lot can be gleaned from the information Nagios provides. A fluctuating network can often be associated with mild, internal hardware or software faults, while a network that has been down for a long and consistent period of time can be attributed to severe internal failures or external attacks. Nagios is also capable of monitoring system load and disk usage, which can directly point out Denial-of-Service attacks. In my project, Nagios correctly identified that several of my nodes were having network latency issues, while others had significant packet loss, and that many services were disabled. When writing recovery scripts, this would help immediately identify the targets to recover.

After a fault is identified, steps can be taken to mitigate said faults, usually through software recoveries or by performing maintenance tasks. For distributed systems, tools like Ansible help orchestrate changes across large groups of hosts, rather than having to repeat procedures one at a time. I did this in my project by restarting pivotal services and flushing traffic control alterations, though in a real-world context this could mean writing an Ansible playbook to have systems pull unbroken Docker containers, rebooting, shutting down for maintenance, or putting systems in lock-down. Having a wide range of community utilities and internal scripting capabilities, Ansible can be very flexible with its utilization.

CPT_S 427 Project – Ansible/Nagios systems management
Dante Meyer

Conclusion

This project has taught me a lot about Vagrant, Ansible, Nagios, and systems like it. For future work that requires distributed systems, I will likely consider using these technologies again as they've proven very useful and intuitive. However, I did run into a lot of issues working with Nagios 4 Core, as it's very evidently an outdated piece of software. The web front-end relies on CGI scripts through an Apache server, contains a lot of mismatched or nonfunctional pages, and the program has very opaque documentation. In the future, I might instead look towards using Nagios XI, the flagship version of the product that they offer with a much cleaner user interface, or another alternative. The primary reason I resorted to Nagios 4 for this project is that creating and destroying virtual machines in rapid succession proved to be faster due to its lightweight nature.